Embeddings

How machines turn words, sentences and images into vectors that capture meaning, and how to use them for search, clustering and RAG.

From the basics to advanced techniques, with explanations and working code.

1. **Foundations**: vectors, one-hot encoding, similarity measures, TF-IDF
2. **Learned embeddings**: Word2Vec, building word vectors yourself, sentence embeddings
3. **Embeddings in practice**: semantic search, clustering, classification, vector databases
4. **Advanced**: approximate nearest neighbour search, quantization, Matryoshka, multimodal, fine-tuning, evaluation

Contents

Part 1: Foundations

1. What is an embedding?
2. Why one-hot encoding is not enough
3. Measuring similarity
4. Bag of words and TF-IDF

Part 2: Learned embeddings

5. The idea behind Word2Vec
6. Build word embeddings yourself, from scratch
7. The limit of word embeddings: one vector per word
8. Sentence embeddings

Part 3: Embeddings in practice

9. Semantic search from scratch
10. Clustering and visualising
11. Embeddings as features for classification
12. Vector databases

Part 4: Advanced topics

13. Approximate nearest neighbour (ANN) search
14. Shrinking embeddings: quantization and Matryoshka
15. Choosing an embedding model
16. Multimodal embeddings: text and images in one space
17. Fine-tuning an embedding model on your own data
18. Evaluating retrieval quality
19. Common pitfalls

Summary and interview questions

Cheat sheet

Interview questions to practise

Part 1: Foundations

1. What is an embedding?

An **embedding** is a list of numbers (a vector) that represents a piece of data, such as a word, a sentence, an image or a product, so that **similar things get similar numbers**.

> **In plain words:** think of GPS coordinates. Dublin is (53.3, -6.3) and Belfast is (54.6, -5.9). The numbers alone mean nothing, but cities that are close in the world are close in the numbers. An embedding does the same for *meaning*: "dog" and "puppy" end up close together, while "dog" and "invoice" end up far apart.

A typical sentence embedding has 384 to 3,072 numbers. You can't picture that many dimensions, but the maths works the same as in 2D: distances and angles between vectors tell you how related two things are.

Embeddings power a huge part of modern AI:

Application	How embeddings are used
Semantic search	Find documents whose vectors are closest to the query's vector
RAG (retrieval-augmented generation)	Retrieve relevant context for an LLM to answer from
Recommendations	Suggest items whose vectors are close to what a user liked
Clustering	Group similar support tickets, reviews or news articles
Classification	Use vectors as features for a simple classifier
Duplicate detection	Flag near-identical questions, products or images
Inside every LLM	The first layer of a Transformer turns each token into an embedding

2. Why one-hot encoding is not enough

Machine learning models need numbers. The simplest way to turn words into numbers is **one-hot encoding**: give each word its own position in a vector, set that position to 1 and everything else to 0.

Python

Tested: output below is real

```python
import numpy as np

vocab = ["cat", "dog", "puppy", "invoice"]
one_hot = {word: np.eye(len(vocab))[i] for i, word in enumerate(vocab)}

for word, vec in one_hot.items():
    print(f"{word:8s} {vec}")

print("\ndog . puppy   =", one_hot["dog"] @ one_hot["puppy"])
print("dog . invoice =", one_hot["dog"] @ one_hot["invoice"])
```

Output

```
cat      [1. 0. 0. 0.]
dog      [0. 1. 0. 0.]
puppy    [0. 0. 1. 0.]
invoice  [0. 0. 0. 1.]

dog . puppy   = 0.0
dog . invoice = 0.0
```

Both dot products are 0. One-hot vectors say every pair of different words is equally unrelated, so "dog" is as far from "puppy" as from "invoice". They also grow with the vocabulary: 50,000 words means 50,000-dimensional vectors that are almost all zeros.

Embeddings fix both problems. They are **dense** (every number carries information), **short** (hundreds of dimensions, not tens of thousands) and **meaningful** (geometry reflects similarity).

3. Measuring similarity

Once things are vectors, you need a way to compare them. There are three common measures.

Dot product: multiply matching numbers and add them up. Large when vectors point the same way *and* are long.

Cosine similarity: the dot product divided by both lengths, which measures only the *angle* between vectors. It ranges from -1 (opposite) through 0 (unrelated) to 1 (same direction). This is the most common choice for text embeddings.

Euclidean distance: the straight-line distance between the two points. Smaller means more similar.

Python — Tested: output below is real

```python
import numpy as np

def cosine_similarity(a, b):
    return (a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))

def euclidean_distance(a, b):
    return np.linalg.norm(a - b)

# Toy 3-D "embeddings": [animal-ness, size, formality]
dog     = np.array([0.9, 0.4, 0.1])
puppy   = np.array([0.8, 0.1, 0.1])
invoice = np.array([0.0, 0.2, 0.9])

for name, v in [("puppy", puppy), ("invoice", invoice)]:
    print(f"dog vs {name:8s} cosine={cosine_similarity(dog, v):.3f}  "
          f"euclidean={euclidean_distance(dog, v):.3f}  dot={dog @ v:.3f}")
```

Output

```
dog vs puppy    cosine=0.957  euclidean=0.316  dot=0.770
dog vs invoice  cosine=0.186  euclidean=1.221  dot=0.170
```

> **Key idea:** if every vector is **normalised** to length 1, the dot product *equals* cosine similarity, and ranking by Euclidean distance gives the same order as ranking by cosine. That's why many systems normalise embeddings once and then use the fast dot product everywhere.

Python — Tested: output below is real

```python
def normalize(x):
    return x / np.linalg.norm(x, axis=-1, keepdims=True)

d, p = normalize(dog), normalize(puppy)
print("length after normalising:", np.linalg.norm(d).round(3))
print("dot product of unit vectors:", round(float(d @ p), 3))
print("cosine similarity:          ", round(float(cosine_similarity(dog, puppy)), 3))
```

Output

```
length after normalising: 1.0
dot product of unit vectors: 0.957
cosine similarity:           0.957
```

4. Bag of words and TF-IDF

Before neural embeddings, documents were represented by counting words. A **bag of words** vector counts how often each vocabulary word appears. **TF-IDF** (term frequency × inverse document frequency) improves on this by down-weighting words that appear everywhere, like "the", and up-weighting rare, distinctive words.

Python

Tested: output below is real

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity as sk_cosine

docs = [
    "The striker scored a late goal to win the match",
    "The team won the football match with a late goal",
    "Preheat the oven and bake the bread for thirty minutes",
    "Knead the dough, then bake the bread until golden",
]

tfidf = TfidfVectorizer(stop_words="english")
X = tfidf.fit_transform(docs)          # sparse matrix: 4 docs x vocabulary
print("Matrix shape:", X.shape)
print("Vocabulary sample:", list(tfidf.get_feature_names_out()[:8]))
print("\nDocument similarity matrix:")
print(sk_cosine(X).round(2))
```

Output

```
Matrix shape: (4, 18)
Vocabulary sample: ['bake', 'bread', 'dough', 'football', 'goal', 'golden', 'knead', 'late']

Document similarity matrix:
[[1.   0.38 0.   0.  ]
 [0.38 1.   0.   0.  ]
 [0.   0.   1.   0.26]
 [0.   0.   0.26 1.  ]]
```

The two football sentences are similar to each other, and so are the two baking sentences. TF-IDF is fast, needs no training data and is still widely used for **keyword search**. You'll see it again in the RAG guide as part of *hybrid search*.

> **Watch out:** TF-IDF only matches **exact words**. "The striker scored" and "the forward found the net" share no words, so their TF-IDF similarity is 0 even though they mean the same thing. Learned embeddings solve this.

Part 2: Learned embeddings

5. The idea behind Word2Vec

> "You shall know a word by the company it keeps." (J. R. Firth, 1957)

This is the **distributional hypothesis**: words that appear in similar contexts have similar meanings. "Coffee" and "tea" both appear near "cup", "drink", "hot" and "morning", so a model that predicts context should give them similar vectors.

Word2Vec (Mikolov et al., 2013) trains a small neural network on exactly this task:

- **Skip-gram** predicts the surrounding words from the centre word.
- **CBOW** (continuous bag of words) predicts the centre word from the surrounding words.

The network's prediction is thrown away. What we keep is the weight matrix of its first layer: one row per word, and those rows are the embeddings.

The famous result is that directions in the space carry meaning:

`vector("king") − vector("man") + vector("woman") ≈ vector("queen")`

Python — Needs: pip install gensim

```python
import gensim.downloader as api

# Pre-trained GloVe vectors (a close cousin of Word2Vec), about 66 MB, downloaded once
wv = api.load("glove-wiki-gigaword-50")

print(wv.most_similar("dublin", topn=5))
print(wv.most_similar(positive=["king", "woman"], negative=["man"], topn=3))
# [('queen', 0.85...), ...]

# Train your own Word2Vec on tokenised sentences
from gensim.models import Word2Vec
sentences = [["the", "cat", "sat", "on", "the", "mat"], ["the", "dog", "sat", "on", "the", "rug"]]
model = Word2Vec(sentences=sentences, vector_size=50, window=3, min_count=1, sg=1, epochs=200)
print(model.wv["cat"][:5])
```

6. Build word embeddings yourself, from scratch

You can see the distributional hypothesis at work with nothing but NumPy. This is the classic **count-based** approach, and it's closely related to what Word2Vec learns:

1. Remove filler words such as "the" and "a", then count how often each word appears near each other word (a **co-occurrence matrix**).
2. Convert counts to **PPMI** (positive pointwise mutual information), which highlights word pairs that appear together more than chance.
3. Compress the matrix with **SVD** (singular value decomposition) into a few dense dimensions. Those dimensions are your embeddings.

Python

Tested: output below is real

```python
import numpy as np

corpus = [
    "the king rules the kingdom", "the queen rules the kingdom",
    "the king wears a golden crown", "the queen wears a golden crown",
    "the cat is a lazy pet", "the dog is a loyal pet",
    "the cat sleeps at home", "the dog sleeps at home",
    "i eat pizza for dinner", "i eat pasta for dinner",
    "pizza with cheese is tasty", "pasta with cheese is tasty",
]
filler = {"the", "a", "i", "is", "for", "with", "at"}
tokens = [[w for w in s.split() if w not in filler] for s in corpus]
vocab = sorted({w for sent in tokens for w in sent})
idx = {w: i for i, w in enumerate(vocab)}

# 1. Co-occurrence counts within a window of 2 words either side
C = np.zeros((len(vocab), len(vocab)))
for sent in tokens:
    for i, w in enumerate(sent):
        for j in range(max(0, i - 2), min(len(sent), i + 3)):
            if i != j:
                C[idx[w], idx[sent[j]]] += 1

# 2. Positive pointwise mutual information
total = C.sum()
p_w = C.sum(axis=1, keepdims=True) / total
p_c = C.sum(axis=0, keepdims=True) / total
with np.errstate(divide="ignore"):
    pmi = np.log((C / total) / (p_w * p_c))
ppmi = np.maximum(pmi, 0)

# 3. SVD: keep the top 6 dimensions as dense embeddings
U, S, Vt = np.linalg.svd(ppmi)
word_vectors = U[:, :6] * S[:6]
word_vectors /= np.linalg.norm(word_vectors, axis=1, keepdims=True)

def most_similar(word, k=3):
    sims = word_vectors @ word_vectors[idx[word]]
    best = np.argsort(-sims)
    return [(vocab[i], round(float(sims[i]), 2)) for i in best if vocab[i] != word][:k]

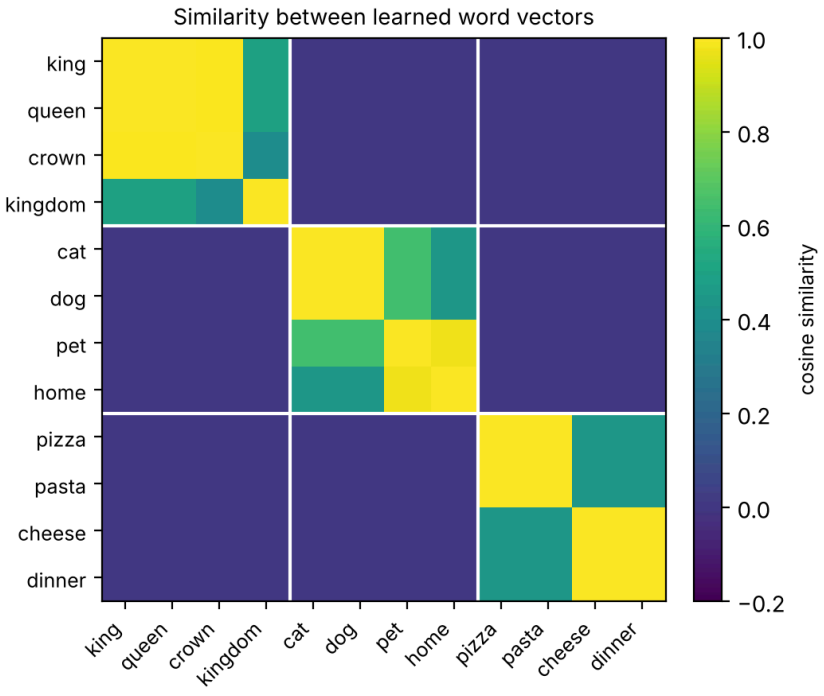
print(f"{len(vocab)} words, {word_vectors.shape[1]} dimensions each\n")
for w in ["king", "cat", "pizza"]:
    print(f"{w:6s} -> {most_similar(w)}")
```

Output

```
20 words, 6 dimensions each

king   -> [('queen', 1.0), ('crown', 0.99), ('kingdom', 0.5)]
cat    -> [('dog', 1.0), ('pet', 0.64), ('loyal', 0.51)]
pizza  -> [('pasta', 1.0), ('tasty', 0.45), ('cheese', 0.45)]
```

The model never saw a definition, yet "king" lands next to "queen", "cat" next to "dog" and "pizza" next to "pasta", purely from shared context. The other neighbours make sense too: "crown" for king, "pet" for cat, "cheese" for pizza.



Bright squares along the diagonal show three groups: royalty, animals and food. Words inside a group have similar vectors, learned only from which words appear near each other.

7. The limit of word embeddings: one vector per word

Word2Vec and GloVe give each word **one fixed vector**. But many words change meaning with context:

- "I deposited cash at the **bank**."
- "We had a picnic on the river **bank**."

A static embedding has to average both meanings into one vector. **Contextual embeddings** from Transformer models (BERT and its descendants) fix this. They read the whole sentence, so "bank" gets a different vector in each sentence. The Transformers guide explains how attention makes this possible.

8. Sentence embeddings

For search, RAG and clustering, you usually want **one vector per sentence or paragraph**, not per word. A sentence embedding model is a Transformer encoder that reads the text and **pools** its token vectors (usually by averaging them) into a single vector. Models such as Sentence-BERT are then trained on pairs of similar and dissimilar sentences, so that sentences with the same meaning land close together, even with no shared words.

Option A: open-source models with sentence-transformers

Python — Needs: pip install sentence-transformers

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")   # small, fast, 384 dimensions; runs on a laptop CPU

sentences = [
    "The striker scored a late winner",
    "The forward found the net in the final minute",
    "Bake the bread until golden",
]
embeddings = model.encode(sentences, normalize_embeddings=True)
print(embeddings.shape)                # (3, 384)

print(model.similarity(embeddings, embeddings))
# Sentences 0 and 1 score high even though they share almost no words.
```

Option B: hosted embedding APIs

Python — Needs: pip install openai

```python
from openai import OpenAI

client = OpenAI()   # reads OPENAI_API_KEY from the environment

response = client.embeddings.create(
    model="text-embedding-3-small",
    input=["The striker scored a late winner", "Bake the bread until golden"],
)
vectors = [item.embedding for item in response.data]
print(len(vectors), len(vectors[0]))    # 2 1536
```

	Open-source (sentence-transformers)	Hosted API (OpenAI, Gemini, Cohere, Voyage...)
Cost	Free; you pay for your own compute	Pay per token (usually cheap)
Privacy	Data never leaves your machine	Data is sent to the provider
Quality	Good to excellent, depending on model	Usually excellent
Setup	Download a model, manage hardware	One API call
Offline use	Yes	No

> **Watch out:** embeddings from **different models are not compatible**. You must embed your documents and your queries with the *same* model. If you switch models, re-embed everything.

Part 3: Embeddings in practice

The examples in this part use **LSA embeddings** (TF-IDF followed by SVD, the same compression trick as Section 6) so they run anywhere without downloads. In a real project you'd swap in a sentence-transformers or API model. The code around it stays the same.

9. Semantic search from scratch

Semantic search has three steps: **embed every document once**, **embed the query**, then **return the documents with the highest similarity**.

Python

Tested: output below is real

```python
import numpy as np
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import make_pipeline

documents = [
    # sport
    "The team scored a late goal to win the league match",
    "Fans cheered as the team won the match with a late goal",
    "The coach praised the players after the league match",
    "The players and the coach celebrated the league title with the fans",
    # cooking
    "Bake the bread in the oven until the crust is golden",
    "Roast the vegetables in the oven with olive oil and garlic",
    "Simmer the tomato sauce, then bake the pasta in the oven",
    "Whisk the eggs and flour, then bake the cake in the oven",
    # technology
    "The new phone has a faster chip and a longer battery life",
    "Update the phone software to fix the battery bug",
    "The laptop software update fixed a security bug",
    "Engineers made the laptop chip faster and the battery last longer",
]

embedder = make_pipeline(TfidfVectorizer(stop_words="english"),
                         TruncatedSVD(n_components=4, random_state=0))
doc_vectors = embedder.fit_transform(documents)
doc_vectors /= np.linalg.norm(doc_vectors, axis=1, keepdims=True)

def search(query, k=3):
    q = embedder.transform([query])[0]
    q /= np.linalg.norm(q)
    scores = doc_vectors @ q
    top = np.argsort(-scores)[:k]
    return [(round(float(scores[i]), 2), documents[i]) for i in top]

for query in ["who won the match", "how long should I roast", "my phone battery"]:
    print(f"Query: {query}")
    for score, doc in search(query):
        print(f"  {score:5.2f}  {doc}")
```

Output

```
Query: who won the match
   1.00  The team scored a late goal to win the league match
   1.00  Fans cheered as the team won the match with a late goal
   1.00  The coach praised the players after the league match
Query: how long should I roast
   1.00  Whisk the eggs and flour, then bake the cake in the oven
   1.00  Simmer the tomato sauce, then bake the pasta in the oven
   1.00  Roast the vegetables in the oven with olive oil and garlic
Query: my phone battery
   0.90  Engineers made the laptop chip faster and the battery last longer
   0.89  The new phone has a faster chip and a longer battery life
   0.70  Update the phone software to fix the battery bug
```

Each query returns three documents from the right topic. Look closely at the second query. The only useful word in it is "roast", which appears in just one document. Keyword search (TF-IDF) would score the other eleven documents 0. Yet the cake and pasta recipes score just as highly as the roast document itself, because the SVD learned that "roast" appears alongside "oven", and "oven" appears alongside "bake". That's the step from keyword matching towards meaning.

> **Key idea:** the document vectors are computed **once** and stored. Each search embeds only the short query, then does one matrix multiplication. That's why vector search scales to millions of documents.

10. Clustering and visualising

Because similar texts have nearby vectors, ordinary clustering algorithms such as **k-means** can group documents by topic without any labels. This is useful for organising support tickets, discovering themes in reviews, or deduplicating content.

Python — Tested: output below is real

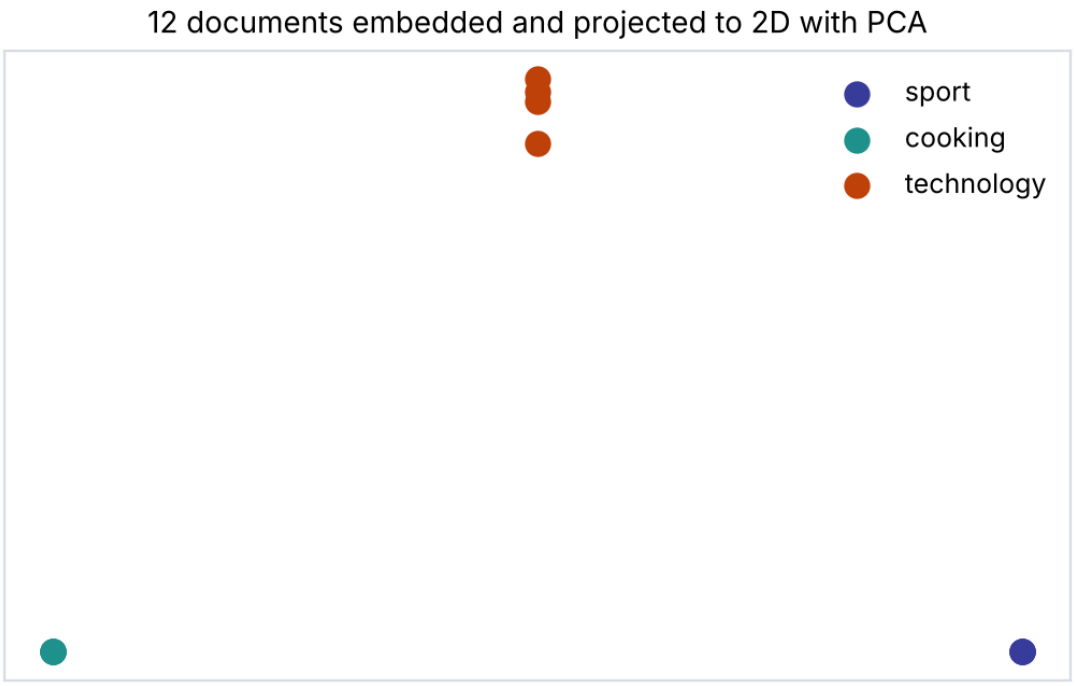
```python
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, n_init=10, random_state=0).fit(doc_vectors)
for cluster in range(3):
    members = [documents[i][:48] for i in np.where(kmeans.labels_ == cluster)[0]]
    print(f"Cluster {cluster}:")
    for m in members:
        print(f"   - {m}...")
```

Output

```
Cluster 0:
   - Bake the bread in the oven until the crust is go...
   - Roast the vegetables in the oven with olive oil ...
   - Simmer the tomato sauce, then bake the pasta in ...
   - Whisk the eggs and flour, then bake the cake in ...
Cluster 1:
   - The new phone has a faster chip and a longer bat...
   - Update the phone software to fix the battery bug...
   - The laptop software update fixed a security bug...
   - Engineers made the laptop chip faster and the ba...
Cluster 2:
   - The team scored a late goal to win the league ma...
   - Fans cheered as the team won the match with a la...
   - The coach praised the players after the league m...
   - The players and the coach celebrated the league ...
```

To *see* high-dimensional embeddings, project them to 2D with **PCA** (fast, linear), **t-SNE** or **UMAP** (non-linear, better at showing clusters, but distances between clusters aren't meaningful).



The documents separate cleanly by topic. Each point is one document; colours show the true topic, which the clustering recovered without labels.

11. Embeddings as features for classification

A powerful, cheap pattern: **embed your texts, then train a simple classifier on the vectors.** Because the embedding model has already captured meaning, a logistic regression on top often matches a much larger fine-tuned model, using only a few hundred labelled examples.

Python — Tested: output below is real

```python
from sklearn.linear_model import LogisticRegression

labels = ["sport"] * 4 + ["cooking"] * 4 + ["tech"] * 4
clf = LogisticRegression(max_iter=1000).fit(doc_vectors, labels)

new_texts = ["The fans waited for the players before the match",
             "Roast the garlic and bake the bread",
             "The software update drains my laptop battery"]
new_vectors = embedder.transform(new_texts)
new_vectors /= np.linalg.norm(new_vectors, axis=1, keepdims=True)
for text, label in zip(new_texts, clf.predict(new_vectors)):
    print(f"{label:8s} <- {text}")
```

Output

```
sport    <- The fans waited for the players before the match
cooking  <- Roast the garlic and bake the bread
tech     <- The software update drains my laptop battery
```

12. Vector databases

A Python list and a matrix multiplication work for thousands of vectors. For millions, plus filtering, updates and persistence, you use a **vector database** or **vector index**.

Tool	Type	Good for
FAISS (Meta)	Library	Fastest local search, billions of vectors, full control
Chroma	Embedded database	Prototypes and small apps; stores text and metadata too
pgvector	PostgreSQL extension	Teams already on Postgres who want SQL plus vectors
Qdrant, Weaviate, Milvus	Vector database servers	Production: filtering, scaling, replication
Pinecone	Managed service	No infrastructure to run

FAISS: exact search

Python — Needs: pip install faiss-cpu sentence-transformers

```python
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
docs = ["The striker scored a late winner", "Bake the bread until golden", "Update your phone
software"]
vectors = model.encode(docs, normalize_embeddings=True).astype("float32")

index = faiss.IndexFlatIP(vectors.shape[1])   # inner product = cosine for unit vectors
index.add(vectors)

query = model.encode(["football result"], normalize_embeddings=True).astype("float32")
scores, ids = index.search(query, 2)
for score, i in zip(scores[0], ids[0]):
    print(round(float(score), 3), docs[i])

faiss.write_index(index, "docs.faiss")       # save to disk; faiss.read_index() to load
```

Chroma: a database that stores text, vectors and metadata

Python — Needs: pip install chromadb

```python
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection("articles")

# Chroma embeds the text for you (default: all-MiniLM-L6-v2 run locally)
collection.add(
    ids=["a1", "a2", "a3"],
    documents=["The striker scored a late winner", "Bake the bread until golden",
               "Update your phone software"],
    metadatas=[{"topic": "sport"}, {"topic": "food"}, {"topic": "tech"}],
)

results = collection.query(query_texts=["football result"], n_results=2)
print(results["documents"][0])

# Combine vector search with a metadata filter
results = collection.query(query_texts=["recipe"], n_results=1, where={"topic": "food"})
```

Part 4: Advanced topics

13. Approximate nearest neighbour (ANN) search

Exact search compares the query with **every** vector. With 100 million 768-dimensional vectors, that's 77 billion multiplications per query, which is too slow. **ANN indexes** trade a little accuracy for a huge speed-up by only examining the most promising part of the data.

Index	How it works	Trade-off
Flat	Compare with everything	100% accurate, slow at scale
IVF (inverted file)	Cluster vectors; search only the nearest few clusters	Fast; accuracy depends on how many clusters you probe
HNSW (graph)	Navigate a multi-layer "small world" graph from coarse to fine	Very fast and accurate; uses more memory; the default in most vector databases
PQ (product quantization)	Compress vectors into short codes	Huge memory savings; some accuracy loss

The best way to understand IVF is to build it. Below, 20,000 vectors are grouped into 100 clusters. At query time we only scan the vectors in the `n_probe` clusters closest to the query, and measure **recall@10**: how many of the true 10 nearest neighbours we still find.

Python — Tested: output below is real

```python
import numpy as np
from sklearn.cluster import MiniBatchKMeans

rng = np.random.default_rng(0)
centres = rng.normal(size=(50, 64))
data = centres[rng.integers(0, 50, 20_000)] + 0.6 * rng.normal(size=(20_000, 64))
data /= np.linalg.norm(data, axis=1, keepdims=True)
queries = data[rng.choice(20_000, 200, replace=False)] + 0.05 * rng.normal(size=(200, 64))
queries /= np.linalg.norm(queries, axis=1, keepdims=True)

# Ground truth with exact (flat) search
true_top10 = np.argsort(-(queries @ data.T), axis=1)[:, :10]

# Build the IVF index: 100 clusters, each vector stored in its cluster's list
kmeans = MiniBatchKMeans(n_clusters=100, random_state=0, n_init=3).fit(data)
centroids = kmeans.cluster_centers_
inverted_lists = [np.where(kmeans.labels_ == c)[0] for c in range(100)]

def ivf_search(q, k=10, n_probe=5):
    nearest_clusters = np.argsort(-(centroids @ q))[:n_probe]
    candidates = np.concatenate([inverted_lists[c] for c in nearest_clusters])
    best = candidates[np.argsort(-(data[candidates] @ q))[:k]]
    return best, len(candidates)

print("n_probe  recall@10  vectors scanned")
for n_probe in [1, 2, 5, 10, 20]:
    recalls, scanned = [], []
    for q, truth in zip(queries, true_top10):
        found, n = ivf_search(q, n_probe=n_probe)
        recalls.append(len(set(found) & set(truth)) / 10)
        scanned.append(n)
    print(f"{n_probe:7d}  {np.mean(recalls):9.3f}  {np.mean(scanned) / len(data):9.1%}")
```

Output

```
n_probe  recall@10  vectors scanned
      1      0.622       1.1%
      2      0.956       2.1%
      5      1.000       4.8%
     10      1.000       9.4%
     20      1.000      19.5%
```

Scanning a small fraction of the data already finds most of the true neighbours, and raising `n_probe` buys back accuracy at the cost of speed. FAISS, Qdrant and every other vector database expose exactly this dial.

Python — Needs: pip install faiss-cpu

```python
import faiss

d, n_clusters = vectors.shape[1], 1024
quantizer = faiss.IndexFlatIP(d)
ivf = faiss.IndexIVFFlat(quantizer, d, n_clusters, faiss.METRIC_INNER_PRODUCT)
ivf.train(vectors)          # learns the clusters; needs a representative sample
ivf.add(vectors)
ivf.nprobe = 16             # the accuracy/speed dial

hnsw = faiss.IndexHNSWFlat(d, 32, faiss.METRIC_INNER_PRODUCT)   # 32 links per node
hnsw.add(vectors)           # no training step needed
```

14. Shrinking embeddings: quantization and Matryoshka

Memory is often the real bottleneck. 10 million 1,024-dimensional float32 vectors take 41 GB. There are two ways to shrink them.

Quantization stores each number with fewer bits. **int8** uses 1 byte instead of 4 (4× smaller). **Binary** keeps only the sign of each number, 1 bit instead of 32 (32× smaller), and compares vectors with very fast Hamming distance. A common trick is to search the compressed vectors for candidates, then **rescore** the top candidates with the full vectors.

Python — Tested: output below is real

```python
rng = np.random.default_rng(1)
centres = rng.normal(size=(200, 256))      # real embeddings are clustered by topic
docs = (centres[rng.integers(0, 200, 10_000)] + rng.normal(size=(10_000, 256))).astype(np.float32)
docs /= np.linalg.norm(docs, axis=1, keepdims=True)
qs = docs[:100] + 0.02 * rng.normal(size=(100, 256)).astype(np.float32)
truth = np.argsort(-(qs @ docs.T), axis=1)[:, :10]

def recall(pred):
    return np.mean([len(set(p) & set(t)) / 10 for p, t in zip(pred, truth)])

# int8: scale each value to the range -127..127
scale = np.abs(docs).max()
docs_i8 = np.round(docs / scale * 127).astype(np.int8)
pred_i8 = np.argsort(-(qs @ docs_i8.T.astype(np.float32)), axis=1)[:, :10]

# binary: keep only the sign (+1 / -1); more matching signs = more similar
docs_bin = np.where(docs > 0, 1, -1).astype(np.int8)
qs_bin = np.where(qs > 0, 1, -1).astype(np.int8)
matches = qs_bin.astype(np.float32) @ docs_bin.T.astype(np.float32)
pred_bin = np.argsort(-matches, axis=1)[:, :10]

# binary search for 100 candidates, then rescore them with float vectors
cand = np.argsort(-matches, axis=1)[:, :100]
pred_rescored = np.array([c[np.argsort(-(docs[c] @ q))[:10]] for c, q in zip(cand, qs)])

print(f"{'method':22s} {'index size':>10s} {'recall@10':>10s}")
for name, mem, pred in [("float32", docs.nbytes, truth), ("int8", docs_i8.nbytes, pred_i8),
                        ("binary", docs.nbytes / 32, pred_bin),
                        ("binary + rescore", docs.nbytes / 32, pred_rescored)]:
    print(f"{name:22s} {mem / 1e6:8.2f}MB {recall(pred):10.3f}")
```

Output

```
method                 index size  recall@10
float32                   10.24MB      1.000
int8                       2.56MB      0.983
binary                     0.32MB      0.515
binary + rescore           0.32MB      1.000
```

int8 is almost lossless. Binary on its own gets the rough neighbourhood right but scrambles the fine ranking, and rescoring just 100 candidates with the original vectors (which can live on cheaper disk storage) recovers it at a fraction of the memory. Real embedding models usually lose even less than this synthetic example.

Matryoshka embeddings are trained so that the **first** dimensions carry the most information, like nested Russian dolls. You can cut a 1,024-dimensional vector down to 256 dimensions and keep most of its quality. Many modern models support this, including OpenAI's `text-embedding-3` family:

Python — Needs: pip install openai sentence-transformers

```python
# OpenAI: ask for fewer dimensions directly
from openai import OpenAI
client = OpenAI()
client.embeddings.create(model="text-embedding-3-large", input=["some text"], dimensions=256)

# sentence-transformers: truncate any Matryoshka-trained model
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("nomic-ai/nomic-embed-text-v1.5", trust_remote_code=True,
truncate_dim=256)
```

> **Watch out:** only truncate models that were **trained** with Matryoshka loss. Cutting dimensions off an ordinary model badly damages quality. Re-normalise after truncating.

15. Choosing an embedding model

Question	Why it matters
How good is it on tasks like mine?	Check the **MTEB** leaderboard (Massive Text Embedding Benchmark), filtered to your task: retrieval, clustering, classification. Then test on *your* data.
How many dimensions?	More dimensions usually means better quality, but more memory and slower search.
What's the maximum input length?	Text past the limit is silently cut off. Many models stop at 512 tokens.
Which languages?	English-only models do badly on other languages; choose a multilingual model if needed.
Hosted or local?	Cost, privacy, latency and offline needs (see the table in Section 8).
Does it need instructions?	Some models expect prefixes such as `query:` and `passage:`. Read the model card.

16. Multimodal embeddings: text and images in one space

CLIP (OpenAI, 2021) trains an image encoder and a text encoder together on 400 million image–caption pairs, so that an image and its description get similar vectors. This enables **text-to-image search**, image-to-image search and **zero-shot classification**: classifying images into categories the model was never explicitly trained on.

Python — Needs: pip install transformers torch pillow

```python
import torch
from PIL import Image
from transformers import CLIPModel, CLIPProcessor

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

image = Image.open("photo.jpg")
labels = ["a photo of a cat", "a photo of a dog", "a photo of a car"]

inputs = processor(text=labels, images=image, return_tensors="pt", padding=True)
with torch.no_grad():
    outputs = model(**inputs)
probs = outputs.logits_per_image.softmax(dim=1)[0]

for label, p in zip(labels, probs):
    print(f"{p:.1%}  {label}")

# The embeddings themselves, for building an image search engine:
image_vec = outputs.image_embeds    # shape (1, 512)
text_vecs = outputs.text_embeds     # shape (3, 512)
```

17. Fine-tuning an embedding model on your own data

General models can struggle with specialised vocabulary: legal clauses, medical notes, product codes, internal jargon. **Fine-tuning** on pairs of texts that *should* match (a question and its answer, a query and the right document) adapts the space to your domain.

The standard loss is **Multiple Negatives Ranking Loss**. For each (anchor, positive) pair in a batch, it treats every *other* positive in the batch as a negative, so you only need positive pairs.

Python — Needs: pip install sentence-transformers datasets

```python
from datasets import Dataset
from sentence_transformers import SentenceTransformer, SentenceTransformerTrainer, losses

model = SentenceTransformer("all-MiniLM-L6-v2")

train = Dataset.from_dict({
    "anchor":   ["How do I reset my password?", "What is the refund policy?"],
    "positive": ["Go to Settings > Security and choose Reset password.",
                 "Refunds are available within 30 days of purchase."],
})   # in practice: 1,000+ pairs from your own data

loss = losses.MultipleNegativesRankingLoss(model)
trainer = SentenceTransformerTrainer(model=model, train_dataset=train, loss=loss)
trainer.train()
model.save_pretrained("my-finetuned-embedder")
```

18. Evaluating retrieval quality

Never judge an embedding model by eye. Build a small **test set** of queries, each with the IDs of the documents that should be found, and measure:

- **Recall@k**: the fraction of queries where a relevant document appears in the top *k*.

- **MRR** (mean reciprocal rank): the average of 1 / (rank of the first relevant document). It rewards putting the right answer *first*.

Python — Tested: output below is real

```python
test_set = {   # query -> index of the correct document in `documents`
    "late winning goal": 0,
    "celebrating the title": 3,
    "golden bread crust": 4,
    "tomato pasta recipe": 6,
    "security fix for the laptop": 10,
    "longer phone battery life": 8,
}

def evaluate(k=3):
    hits, reciprocal_ranks = 0, []
    for query, correct in test_set.items():
        q = embedder.transform([query])[0]
        q /= np.linalg.norm(q)
        ranking = list(np.argsort(-(doc_vectors @ q)))
        rank = ranking.index(correct) + 1
        hits += rank <= k
        reciprocal_ranks.append(1 / rank)
    print(f"Recall@{k}: {hits / len(test_set):.2f}")
    print(f"MRR:      {np.mean(reciprocal_ranks):.2f}")

evaluate()
```

Output

```
Recall@3: 0.67
MRR:      0.50
```

A score of 0.67 tells you something real: with only 4 dimensions, the model has learned *topics* but not *details*, so it can't tell which cooking document a cooking query means. That's exactly the kind of weakness an evaluation set reveals and eyeballing hides. With a test set like this you can compare models, chunk sizes and index settings **objectively**. Fifty well-chosen queries are far more useful than none.

19. Common pitfalls

- **Mixing models.** Queries and documents must use the same model and version.
- **Forgetting to normalise** when your index uses the dot product.
- **Text longer than the model's limit** is silently cut off. Split it into chunks first (see the RAG guide).
- **Ignoring query/passage prefixes** that some models require.
- **Comparing raw scores across models.** A cosine of 0.6 is high for one model and low for another. Compare rankings, not absolute values.
- **No evaluation set**, so you can't tell whether a change helped.

Summary and interview questions

Cheat sheet

Concept	One-line summary
Embedding	A dense vector where geometric closeness means semantic similarity
One-hot	Sparse, huge, and treats all words as equally different
Cosine similarity	Angle between vectors; equals the dot product for unit vectors
TF-IDF	Weighted word counts; exact keyword matching, no training needed
Word2Vec / GloVe	One static vector per word, learned from context
Contextual embeddings	Transformer-based; the same word gets different vectors in different sentences
Sentence embeddings	One vector per text, from a pooled Transformer trained on similar pairs
Vector database	Stores vectors and finds nearest neighbours quickly, with metadata filters
ANN (IVF, HNSW)	Search a subset of the data; trades a little recall for big speed-ups
Quantization	Store vectors in int8 or binary to save 4–32× memory
Matryoshka	Models whose first dimensions carry the most information, so they can be truncated
CLIP	Images and text in one shared space
Recall@k, MRR	The standard ways to measure retrieval quality

Interview questions to practise

1. **Why use cosine similarity instead of Euclidean distance for text embeddings?** Cosine ignores vector length and compares only direction, which reflects meaning better for text. For normalised vectors the two give the same ranking.
2. **What is the difference between Word2Vec and BERT embeddings?** Word2Vec gives each word one fixed vector. BERT produces contextual vectors, so "bank" differs between "river bank" and "bank account".
3. **How would you build semantic search over 10 million documents?** Chunk the documents, embed them with one model, store them in an ANN index (such as HNSW) in a vector database with metadata, embed each query with the same model, retrieve the top *k*, and evaluate with recall@k on a labelled test set.
4. **Your semantic search misses exact product codes. What do you do?** Add keyword search (BM25) and combine the two (hybrid search), because embeddings blur exact tokens.
5. **How do you reduce the memory of a large vector index?** Use quantization (int8, binary or product quantization), Matryoshka truncation, or both, with rescoring of the top candidates.
6. **When would you fine-tune an embedding model?** When the domain has specialised vocabulary and a general model scores poorly on your own retrieval test set, and you can collect at least a few hundred relevant query–document pairs.